

Gaussian Processes

Advanced Statistical Inference

Simone Rossi

Where are we?

We have seen:

1. Bayesian linear regression
2. Approximate inference
 - Markov chain Monte Carlo
 - Variational inference
 - Laplace approximation
3. Bayesian logistic regression (for classification)
4. **Today:** Gaussian processes

Introduction

- Gaussian processes (GPs) are a flexible and powerful tool for regression (and classification).
- Linear models require us to specify a set of basis functions and their coefficients.
 - Polynomials? Trigonometric functions? etc?
- Can we use Bayesian inference to let the data decide?
- Gaussian processes work implicitly with an infinite-dimensional set of basis functions and learn a probabilistic combination of them.

Gaussian Processes (GPs)

Gaussian processes can be explained in two ways:

- **Weight-space view:**
 - Understand GPs as a Bayesian linear regression model with *implicit* basis functions.
 - Since the basis functions are implicit, we can even work with **infinite-dimensional** feature spaces.
- **Function-space view:**
 - Understand GPs as a distribution over functions.
 - There are no parameters in this view, only functions, so we can *implicitly* work with infinite-dimensional parameter spaces.

Function-space view of Gaussian Processes

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2} |\boldsymbol{\Sigma}|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right)$$

Why use Gaussian?

Gaussian **distribution** have several nice properties that make them attractive:

- Sums of Gaussians are Gaussian.
- Marginals of multivariate Gaussians are Gaussian.
- Conditionals of Gaussians are Gaussian.

Conditioning a Gaussian distribution

$$\mathbf{f} = \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{bmatrix}, \begin{bmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{bmatrix} \right)$$

Then

$$p(\mathbf{f}_2 \mid \mathbf{f}_1) = \mathcal{N}(\boldsymbol{\mu}_2 + \boldsymbol{\Sigma}_{21} \boldsymbol{\Sigma}_{11}^{-1} (\mathbf{f}_1 - \boldsymbol{\mu}_1), \boldsymbol{\Sigma}_{22} - \boldsymbol{\Sigma}_{21} \boldsymbol{\Sigma}_{11}^{-1} \boldsymbol{\Sigma}_{12})$$

which simplifies if $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2 = \mathbf{0}$,

$$p(\mathbf{f}_2 | \mathbf{f}_1) = \mathcal{N}(\Sigma_{21}\Sigma_{11}^{-1}\mathbf{f}_1, \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12})$$

💡 Tip

This is sometimes referred to as the **Bayes' rule for Gaussian distributions**.

Gaussian Processes as a distribution over functions

- We can think of a Gaussian process as an infinite-dimensional generalization of a multi-variate Gaussian distribution.
- ... or as an infinite-dimensional distribution over functions.
- All we need to do is to change indexes

Stochastic processes

A stochastic process is a collection of random variables indexed by some variable $x \in \mathcal{X}$.

$$\{\mathbf{f}(x) : x \in \mathcal{X}\}$$

Usually $\mathbf{f}(x) \in \mathbb{R}$ and $\mathcal{X} = \mathbb{R}^D$. So, $\mathbf{f} : \mathbb{R}^D \rightarrow \mathbb{R}$.

...

For a set of inputs $\mathbf{X} = \{x_1, \dots, x_N\}$, the corresponding random variables $\{\mathbf{f}_1, \dots, \mathbf{f}_N\}$ with $\mathbf{f}_i = \mathbf{f}(x_i)$ have a joint distribution $p(\mathbf{f}(x_1), \dots, \mathbf{f}(x_N))$.

...

A **Gaussian process** is a stochastic process such that any finite subset of the random variables has a joint Gaussian distribution.

$$(\mathbf{f}(x_1), \dots, \mathbf{f}(x_N)) \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

We write $\mathbf{f} \sim \mathcal{GP}$ to denote that $\mathbf{f}$ is a Gaussian process.

Gaussian Processes: mean and covariance functions

To specify a Gaussian distribution, we need to define a mean vector $\boldsymbol{\mu}$ and a covariance matrix $\boldsymbol{\Sigma}$.

$$\boldsymbol{a} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

To specify a Gaussian process, we need to define a mean **function** $\mu(\boldsymbol{x})$ and a covariance **function** $k(\boldsymbol{x}, \boldsymbol{x}')$.

$$f \sim \mathcal{GP}(\mu(\cdot), k(\cdot, \cdot))$$

- The mean function $\mu(\boldsymbol{x}) = \mathbb{E}[f(\boldsymbol{x})]$.
- The covariance function $k(\boldsymbol{x}, \boldsymbol{x}') = \mathbb{E}[(f(\boldsymbol{x}) - \mu(\boldsymbol{x}))(f(\boldsymbol{x}') - \mu(\boldsymbol{x}'))]$.

Take two points $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$, their function values $f_0 = f(\boldsymbol{x}_0)$ and $f_1 = f(\boldsymbol{x}_1)$ are jointly Gaussian distributed with $p(f_0, f_1)$.

Mean function

We are free to choose the mean and the covariance functions however we like, subject to some constraints.

- The mean function can be used to encode prior knowledge about the mean of the function we are trying to model.
- The mean function $\mu(\boldsymbol{x})$ is often set to zero or constant.

Covariance function

The covariance function $k(\boldsymbol{x}, \boldsymbol{x}')$ is also known as the kernel function.

The covariance function encodes our prior beliefs about the **family of functions** we are trying to model.

...

Requirements for a valid covariance function:

- k must be a positive semi-definite function.

- Given locations $\mathbf{x}_1, \dots, \mathbf{x}_N$, the covariance matrix $\mathbf{K}$ with entries $k_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ must be positive semi-definite.

We often assume k is a function of the difference between the inputs: $k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x} - \mathbf{x}')$, which is known as a **stationary kernel**.

Squared Exponential Kernel

RBf kernel or squared exponential kernel:

$$k(\mathbf{x}, \mathbf{x}') = \alpha \exp\left(-\frac{1}{2\lambda^2}(\mathbf{x} - \mathbf{x}')^\top(\mathbf{x} - \mathbf{x}')\right) = \alpha \exp\left(-\frac{1}{2\lambda^2}\|\mathbf{x} - \mathbf{x}'\|^2\right)$$

Defines smooth functions, where the function values are highly correlated for nearby inputs.

Hyperparameters:

- α is the amplitude
- λ is the length scale

Predictions with Gaussian Processes

Given a set of training inputs $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and outputs $\mathbf{f} = \{f_1, \dots, f_N\}$, we want to make predictions for a new input $\mathbf{x}_\star$.

The joint distribution of the training outputs $\mathbf{f}$ and the test output $f_\star$ is Gaussian:

$$\begin{bmatrix} \mathbf{f} \\ f_\star \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K} & \mathbf{k}_\star \\ \mathbf{k}_\star^\top & k_{\star\star} \end{bmatrix}\right)$$

where $\mathbf{K} = k(\mathbf{X}, \mathbf{X})$, $\mathbf{k}_\star = k(\mathbf{X}, \mathbf{x}_\star)$, and $k_{\star\star} = k(\mathbf{x}_\star, \mathbf{x}_\star)$.

...

Because of the Gaussian conditioning property, the distribution of $f_\star$ given $\mathbf{f}$ is also Gaussian:

$$f_\star \mid \mathbf{f} \sim \mathcal{N}(\mathbf{k}_\star^\top \mathbf{K}^{-1} \mathbf{f}, k_{\star\star} - \mathbf{k}_\star^\top \mathbf{K}^{-1} \mathbf{k}_\star)$$

Predictions with Gaussian Processes

$$f_{\star} \mid \mathbf{f} \sim \mathcal{N}(\mathbf{k}_{\star}^{\top} \mathbf{K}^{-1} \mathbf{f}, k_{\star\star} - \mathbf{k}_{\star}^{\top} \mathbf{K}^{-1} \mathbf{k}_{\star})$$

Assumes that the training outputs $\mathbf{f}$ are noise-free. *This is not the assumption we made previously.*

In general, we model the noise in the outputs as:

$$p(\mathbf{y} \mid \mathbf{f}) = \mathcal{N}(\mathbf{y} \mid \mathbf{f}, \sigma^2 \mathbf{I})$$

Predictions with Gaussian Processes with noise

The joint distribution of the training outputs $\mathbf{y}$ and the test output $f_{\star}$ is still Gaussian, but with an additional noise term:

$$\begin{bmatrix} \mathbf{y} \\ f_{\star} \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K} + \sigma^2 \mathbf{I} & \mathbf{k}_{\star} \\ \mathbf{k}_{\star}^{\top} & k_{\star\star} \end{bmatrix}\right)$$

The distribution of $f_{\star}$ given $\mathbf{y}$ is:

$$f_{\star} \mid \mathbf{y} \sim \mathcal{N}(\mathbf{k}_{\star}^{\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}, k_{\star\star} - \mathbf{k}_{\star}^{\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}_{\star})$$

We can also marginalize out the noise to get the distribution of $\mathbf{y}_{\star}$:

$$\mathbf{y}_{\star} \mid \mathbf{y} \sim \mathcal{N}(\mathbf{k}_{\star}^{\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}, k_{\star\star} - \mathbf{k}_{\star}^{\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}_{\star} + \sigma^2)$$

Weight-space view of Gaussian Processes

Review: Bayesian Linear Regression

Recall lecture on [Bayesian Linear Regression](#):

- Modeling the data as a linear combination of the features plus noise:

$$p(\mathbf{y} \mid \mathbf{w}, \mathbf{X}) = \mathcal{N}(\mathbf{y} \mid \mathbf{X} \mathbf{w}, \sigma^2 \mathbf{I})$$

- Prior over the weights:

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \mathbf{I})$$

Review: Bayesian Linear Regression

- Posterior over the weights:

$$p(\mathbf{w} \mid \mathbf{y}, \mathbf{X}) = \mathcal{N}(\mathbf{w} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$$

with

$$\boldsymbol{\Sigma} = \left(\frac{1}{\sigma^2} \mathbf{X}^\top \mathbf{X} + \mathbf{I} \right)^{-1} \quad \text{and} \quad \boldsymbol{\mu} = \sigma^{-2} \boldsymbol{\Sigma} \mathbf{X}^\top \mathbf{y}$$

- Predictive distribution for a new input $\mathbf{x}_\star$:

$$p(y_\star \mid \mathbf{x}_\star, \mathbf{y}, \mathbf{X}) = \mathcal{N}(y_\star \mid \boldsymbol{\mu}^\top \mathbf{x}_\star, \mathbf{x}_\star^\top \boldsymbol{\Sigma} \mathbf{x}_\star + \sigma^2)$$

Introducing basis functions

- Transform the inputs using a set of D basis functions:

$$\mathbf{x} \rightarrow \boldsymbol{\phi}(\mathbf{x}) = \begin{bmatrix} \phi_1(\mathbf{x}) & \cdots & \phi_D(\mathbf{x}) \end{bmatrix}^\top$$

- The functions $\phi_d(\mathbf{x})$ are known as *basis functions*.
- Define:

$$\boldsymbol{\Phi} = \begin{bmatrix} \phi_1(\mathbf{x}_1) & \cdots & \phi_D(\mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ \phi_1(\mathbf{x}_N) & \cdots & \phi_D(\mathbf{x}_N) \end{bmatrix} \in \mathbb{R}^{N \times D}$$

Introducing basis functions

Apply Bayesian linear regression to the transformed inputs:

$$p(\mathbf{w} \mid \mathbf{y}, \mathbf{X}) = \mathcal{N}(\mathbf{w} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$$

where

$$\boldsymbol{\Sigma} = \left(\frac{1}{\sigma^2} \boldsymbol{\Phi}^\top \boldsymbol{\Phi} + \mathbf{I} \right)^{-1} \quad \text{and} \quad \boldsymbol{\mu} = \sigma^{-2} \boldsymbol{\Sigma} \boldsymbol{\Phi}^\top \mathbf{y}$$

- Predictive distribution for a new input $\mathbf{x}_\star$:

$$p(y_\star \mid \mathbf{x}_\star, \mathbf{y}, \mathbf{X}) = \mathcal{N}(y_\star \mid \boldsymbol{\mu}^\top \boldsymbol{\phi}_\star, \boldsymbol{\phi}_\star^\top \boldsymbol{\Sigma} \boldsymbol{\phi}_\star + \sigma^2)$$

where $\boldsymbol{\phi}_\star = \boldsymbol{\phi}(\mathbf{x}_\star)$.

Bayesian Linear Regression as a kernel method

- We are going to show that predictions in Bayesian linear regression can be expressed in terms of scalar products

$$k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^\top \boldsymbol{\phi}(\mathbf{x}')$$

- This is known as the *kernel trick*.
- The kernel function $k(\mathbf{x}, \mathbf{x}')$ is a measure of similarity between two inputs $\mathbf{x}$ and $\mathbf{x}'$.

For a matrix input $\mathbf{X}$, the kernel matrix is defined as

$$\mathbf{K} = \boldsymbol{\Phi} \boldsymbol{\Phi}^\top = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix} \in \mathbb{R}^{N \times N}$$

Intuition of the kernel trick

💡 Woodbury matrix identity

Recall the Woodbury matrix identity:

$$(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}$$

(Do not remember this formula!)

Use the Woodbury matrix identity to rewrite the posterior covariance:

$$\begin{aligned}\Sigma &= \left(\frac{1}{\sigma^2} \Phi^\top \Phi + \mathbf{I} \right)^{-1} \\ &= \mathbf{I} - \Phi^\top \left(\sigma^2 \mathbf{I} + \Phi \Phi^\top \right)^{-1} \Phi\end{aligned}$$

Predictions can be expressed in terms of scalar products

$$p(y_\star \mid \mathbf{x}_\star, \mathbf{y}, \mathbf{X}) = \mathcal{N}(y_\star \mid \boldsymbol{\mu}^\top \phi_\star, \phi_\star^\top \Sigma \phi_\star + \sigma^2)$$

We want to express the predictive mean and variance in terms of scalar products of the basis functions.

Variance:

$$\begin{aligned}\phi_\star^\top \Sigma \phi_\star + \sigma^2 &= \phi_\star^\top \left(\mathbf{I} - \Phi^\top \left(\sigma^2 \mathbf{I} + \Phi \Phi^\top \right)^{-1} \Phi \right) \phi_\star + \sigma^2 \\ &= \phi_\star^\top \phi_\star - \phi_\star^\top \Phi^\top \left(\sigma^2 \mathbf{I} + \Phi \Phi^\top \right)^{-1} \Phi \phi_\star + \sigma^2 \\ &= k_{\star\star} - \mathbf{k}_\star^\top (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}_\star + \sigma^2\end{aligned}$$

where $\mathbf{k}_\star = k(\mathbf{X}, \mathbf{x}_\star)$ and $k_{\star\star} = k(\mathbf{x}_\star, \mathbf{x}_\star)$.

Compare this with the predictive variance in the Gaussian process we have seen [before](#).

Predictions can be expressed in terms of scalar products

Mean:

$$\begin{aligned}
 \boldsymbol{\mu}^\top \boldsymbol{\phi}_\star &= \sigma^{-2} \boldsymbol{\phi}_\star^\top \boldsymbol{\Sigma} \boldsymbol{\Phi}^\top \mathbf{y} \\
 &= \sigma^{-2} \boldsymbol{\phi}_\star^\top \left(\mathbf{I} - \boldsymbol{\Phi}^\top (\sigma^2 \mathbf{I} + \boldsymbol{\Phi} \boldsymbol{\Phi}^\top)^{-1} \boldsymbol{\Phi} \right) \boldsymbol{\Phi}^\top \mathbf{y} \\
 &= \sigma^{-2} \boldsymbol{\phi}_\star^\top \boldsymbol{\Phi}^\top \left(\mathbf{I} - (\sigma^2 \mathbf{I} + \boldsymbol{\Phi} \boldsymbol{\Phi}^\top)^{-1} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top \right) \mathbf{y} \\
 &= \sigma^{-2} \boldsymbol{\phi}_\star^\top \boldsymbol{\Phi}^\top \left(\mathbf{I} - (\mathbf{I} + \sigma^{-2} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top)^{-1} \sigma^{-2} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top \right) \mathbf{y}
 \end{aligned}$$

Apply again the Woodbury matrix identity with $\mathbf{A}, \mathbf{U}, \mathbf{V} = \mathbf{I}$ and $\mathbf{C} = (\sigma^{-2} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top)^{-1}$:

$$\mathbf{I} - (\mathbf{I} + \sigma^{-2} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top)^{-1} \sigma^{-2} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top = (\mathbf{I} + \sigma^{-2} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top)^{-1}$$

continue...

Predictions can be expressed in terms of scalar products

$$\begin{aligned}
 \boldsymbol{\mu}^\top \boldsymbol{\phi}_\star &= \sigma^{-2} \boldsymbol{\phi}_\star^\top \boldsymbol{\Phi}^\top \left(\mathbf{I} - (\mathbf{I} + \sigma^{-2} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top)^{-1} \sigma^{-2} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top \right) \mathbf{y} \\
 &= \sigma^{-2} \boldsymbol{\phi}_\star^\top \boldsymbol{\Phi}^\top (\mathbf{I} + \sigma^{-2} \boldsymbol{\Phi} \boldsymbol{\Phi}^\top)^{-1} \mathbf{y} \\
 &= \boldsymbol{\phi}_\star^\top \boldsymbol{\Phi}^\top (\sigma^2 \mathbf{I} + \boldsymbol{\Phi} \boldsymbol{\Phi}^\top)^{-1} \mathbf{y} \\
 &= \mathbf{k}_\star^\top (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}
 \end{aligned}$$

Again, compare this with the predictive mean in the Gaussian process we have seen [before](#).

Intuition of the kernel trick

You can either to linear regression by working with the feature mappings $\boldsymbol{\phi}(\cdot)$ or with the kernel function $k(\cdot, \cdot)$.

1. Working with $\boldsymbol{\phi}(\cdot)$ costs $O(D^2)$ memory and $O(D^3)$ time.
2. Working with $k(\cdot, \cdot)$ costs $O(N^2)$ memory and $O(N^3)$ time.

If $D \gg N$, the kernel trick is more efficient.

For example, working with $k(\cdot, \cdot)$ allows us to implicitly use infinite-dimensional feature spaces.

RBF kernel has infinite basis functions

One of such examples is the RBF (or squared exponential) kernel we have seen before:

$$k(\mathbf{x}, \mathbf{x}') = \alpha \exp\left(-\frac{1}{2\lambda^2}\|\mathbf{x} - \mathbf{x}'\|^2\right)$$

We can show that the RBF kernel corresponds to an infinite-dimensional feature space.

RBF kernel has infinite basis functions

Suppose $\lambda = 1$ and $\alpha = 1$, and $\mathbf{x} \in \mathbb{R}$.

$$k(x, x') = \exp\left(-\frac{1}{2}(x - x')^2\right) = \exp\left(-\frac{1}{2}x^2\right) \exp\left(-\frac{1}{2}x'^2\right) \exp(xx')$$

Use the Taylor expansion of the exponential function:

$$\exp(xx') = 1 + xx' + \frac{1}{2!}(xx')^2 + \frac{1}{3!}(xx')^3 + \dots$$

Define the infinite-dimensional feature mappings:

$$\mathbf{x} \rightarrow \phi(\mathbf{x}) = \exp\left(-\frac{1}{2}x^2\right) \begin{bmatrix} 1 & x & \frac{1}{\sqrt{2}!}x^2 & \frac{1}{\sqrt{3}!}x^3 & \dots \end{bmatrix}^\top$$

Model selection in Gaussian Processes

Model selection in Gaussian Processes

- In practice, we need to choose the kernel function and its hyperparameters.
 - For example, in the RBF kernel, we need to choose the amplitude α and the length scale λ .
- The choice of the kernel function and its hyperparameters is known as *model selection*.
- We already saw how to do model selection in Bayesian linear regression using the marginal likelihood.
- In Gaussian processes, we can also use the marginal likelihood to perform model selection.

$$p(\mathbf{y}) = \int p(\mathbf{y} | \mathbf{f}) p(\mathbf{f}) d\mathbf{f}$$

Marginal likelihood in Gaussian Processes

Let's be explicit about the hyperparameters $\boldsymbol{\theta}$ of the kernel function $k(\cdot, \cdot)$:

$$p(\mathbf{y} | \boldsymbol{\theta}) = \int p(\mathbf{y} | \mathbf{f}, \boldsymbol{\theta}) p(\mathbf{f} | \boldsymbol{\theta}) d\mathbf{f} = \mathcal{N}(\mathbf{y} | \mathbf{0}, \mathbf{K} + \sigma^2 \mathbf{I})$$

Maximize the marginal (log-)likelihood with respect to the hyperparameters $\boldsymbol{\theta}$:

$$\arg \max_{\boldsymbol{\theta}} \log p(\mathbf{y} | \boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} -\frac{1}{2} \mathbf{y}^\top (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y} - \frac{1}{2} \log \det(\mathbf{K} + \sigma^2 \mathbf{I}) - \frac{N}{2} \log(2\pi)$$

Question: How to optimize the marginal likelihood?

Optimization of the marginal likelihood

- The marginal likelihood is a non-convex function of the hyperparameters $\boldsymbol{\theta}$.
- No closed-form solution to the optimization problem, but we can use gradient-based optimization methods.
- We can use automatic differentiation to compute the gradients of the marginal likelihood with respect to the hyperparameters.

$$\frac{\partial}{\partial \theta_i} \log p(\mathbf{y} | \boldsymbol{\theta}) = \frac{1}{2} \mathbf{y}^\top (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \frac{\partial \mathbf{K}}{\partial \theta_i} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y} - \frac{1}{2} \text{tr} \left((\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \frac{\partial \mathbf{K}}{\partial \theta_i} \right)$$

Remember to guarantee that the hyperparameters are valid (e.g., positive for the length scale).

Why the marginal likelihood works

$$\underbrace{-\frac{1}{2} \mathbf{y}^\top (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}}_{\text{Model fit}} - \underbrace{\frac{1}{2} \log \det(\mathbf{K} + \sigma^2 \mathbf{I}) - \frac{N}{2} \log(2\pi)}_{\text{Model complexity}}$$

The marginal likelihood is a trade-off between the fit to the data and the complexity of the model.

Applications of Gaussian Processes

Time Series Foundation Models

Gaussian processes are a natural choice for modeling time series data, as they can capture complex temporal dependencies and provide uncertainty estimates.

Amazon uses GPs to generate synthetic time series data for pre-training a foundation model for time series forecasting

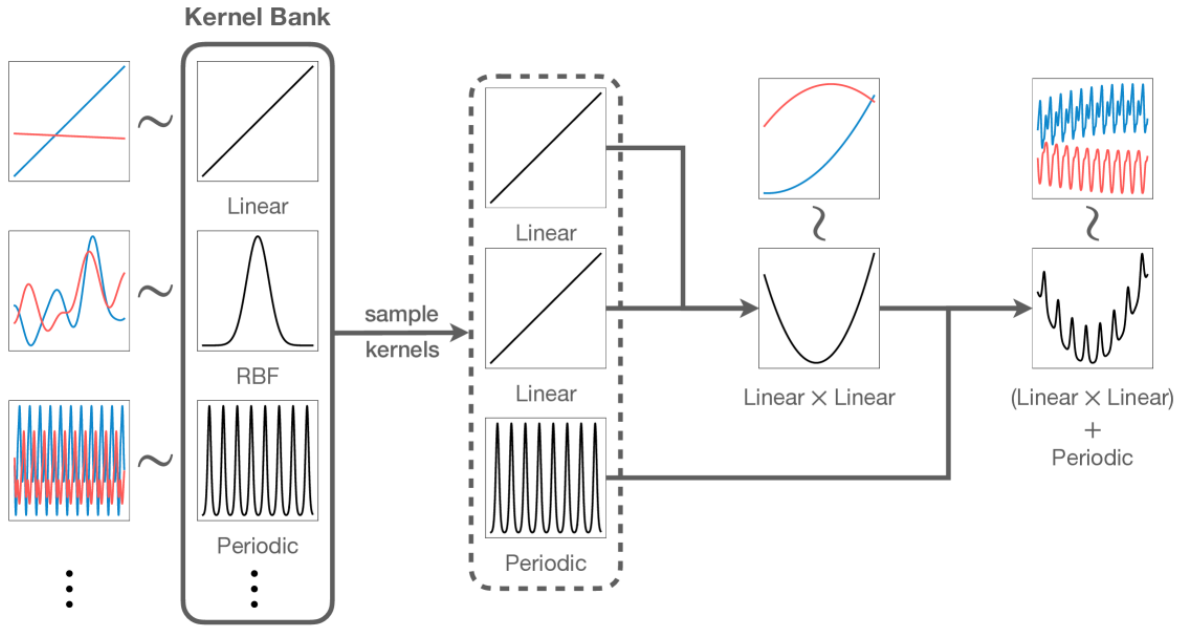
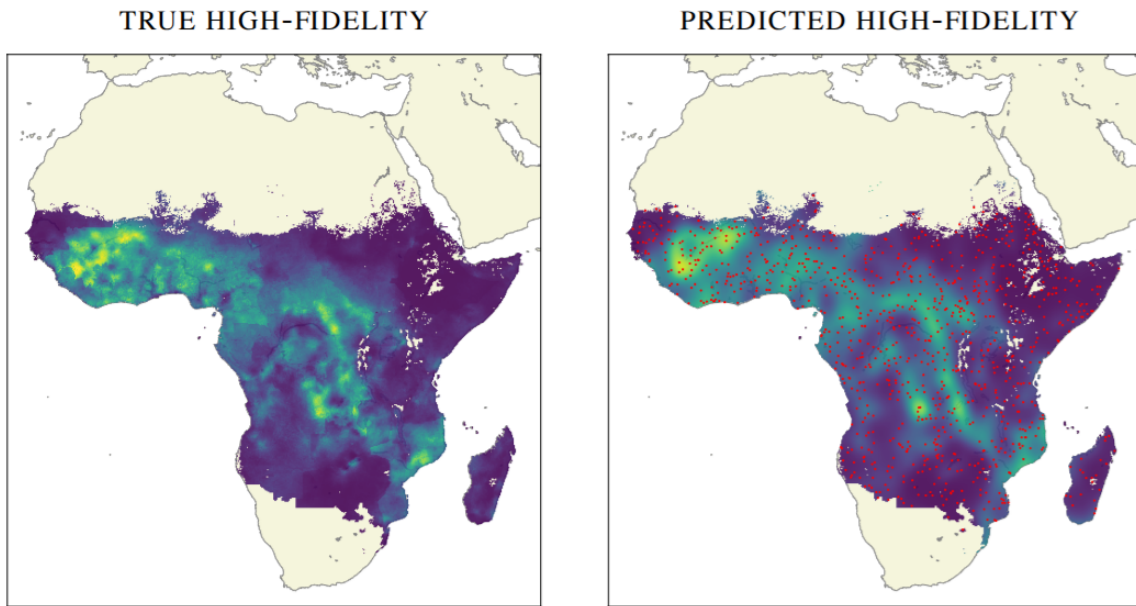


Figure 1: KernelSynth: kernels are sampled from a bank, randomly composed ($\times$ or $+$), and used in a GP prior to generate diverse synthetic time series for pretraining. Ansari et al. (2024).

Spatial Modeling & Science



Geostatistics

- Interpolate geospatial fields: rainfall, mineral deposits, temperature, infection spread, etc.
- Closed-form uncertainty $\rightarrow$ principled gap-filling

Protein Stability Prediction

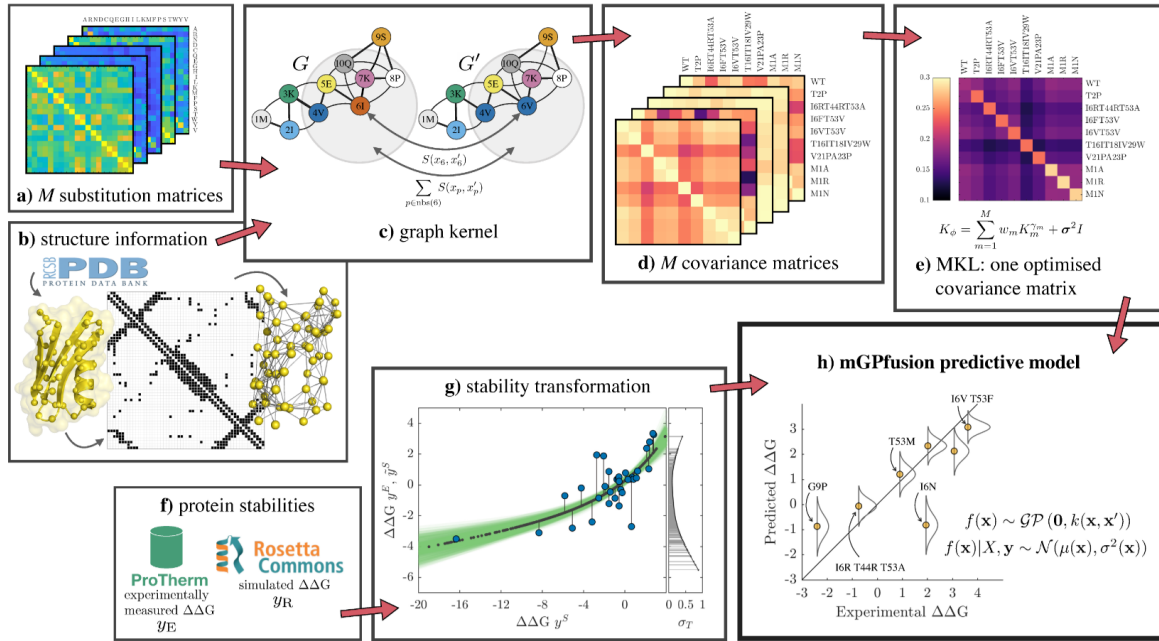


Figure 2: GP surrogate for predicting protein stability changes ($\Delta\Delta G$) upon mutation.

- Measuring protein stability is **slow and costly** — only a few hundred labeled examples
- GPs thrive in the **small-data regime** with principled uncertainty

Challenges in modern Gaussian Processes

Challenges in modern Gaussian Processes

- **Scalability:** Gaussian processes have $\mathcal{O}(N^3)$ complexity in the number of data points N .
 - Solutions: subsets of data, kernel approximations, inducing points, random Fourier features, etc.
- **Non-Gaussian likelihoods:** Exact Gaussian processes assume Gaussian likelihoods, but many real-world datasets have non-Gaussian likelihoods.
 - Solutions: approximate inference

...

Modern Gaussian processes introduce **two** very different types of approximations:

- **Model approximation:** for scalability
- **Inference approximation:** for non-Gaussian likelihoods

Scalable Gaussian Processes

Motivation:

- Gaussian processes have $\mathcal{O}(N^3)$ complexity in the number of data points N .
- Above $N \approx 10,000$ data points, exact Gaussian processes become impractical.

Solution: Find approximations to scale Gaussian processes to large datasets.

Two main approaches:

- Sparse Gaussian processes with inducing points (inspired by the function-space view)
- Kernel approximations using random Fourier features (inspired by the weight-space view)

Inducing points

Idea: Augment the model with a set of $M \ll N$ inducing points $\mathbf{Z} = \{z_1, \dots, z_M\}$ with corresponding function values $\mathbf{u} = \{u_1, \dots, u_M\}$.

Problem: Somehow, solve the inference problem in this augmented model (which is still Gaussian) on a lower computational budget than the original model.

Is this possible? Is it still possible to get exact inference in this augmented model? At what cost?

...

Yes, the predictive distribution for a new input $\mathbf{x}_\star$ is:

$$p(y_\star | \mathbf{y}, \mathbf{X}, \mathbf{Z}) = \mathcal{N}\left(y_\star | \mathbf{k}_{\star\mathbf{Z}}^\top \mathbf{Q}_{\mathbf{ZZ}}^{-1} \mathbf{K}_{\mathbf{ZX}} (\mathbf{\Lambda} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}, k_{\star\star} - \mathbf{k}_{\star\mathbf{Z}}^\top (\mathbf{K}_{\mathbf{ZZ}}^{-1} - \mathbf{Q}_{\mathbf{ZZ}}^{-1}) \mathbf{k}_{\star\mathbf{Z}} + \sigma^2\right)$$

where $\mathbf{Q}_{\mathbf{ZZ}} = \mathbf{K}_{\mathbf{ZZ}} + \mathbf{K}_{\mathbf{ZX}} (\mathbf{\Lambda} + \sigma^2 \mathbf{I})^{-1} \mathbf{K}_{\mathbf{XZ}}$.

Yes, the computational cost is $\mathcal{O}(NM^2)$, instead of $\mathcal{O}(N^3)$.

Gaussian Processes classification

- Gaussian processes can be used for classification problems.
- Model the latent function $f(\mathbf{x})$ as a Gaussian process and apply sigmoid function to get the probability of the class label:

$$p(y = 1 \mid \mathbf{x}) = \sigma(f(\mathbf{x}))$$

- Equivalent to assume the Bernoulli likelihood...

$$p(y \mid f) = \text{Bern}(y \mid \sigma(f))$$

- ... and factorization

$$p(y \mid f) = \prod_{n=1}^N p(y_n \mid f_n)$$

- The posterior predictive distribution is not Gaussian anymore and intractable.

Approximate inference for Gaussian Processes classification

As we did for [Bayesian logistic regression](#), we need to use approximate inference methods.

- Laplace approximation:
 1. Find the mode of the posterior distribution using optimization methods.
 2. Compute the Hessian at the mode.
 3. Approximate the posterior distribution as a Gaussian with the mode and the Hessian.
- Variational inference